Type signature
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-800004.4

e.g. check type in GHCI

```
ghci> :type 1
1 :: Num a => a
ghci> :t 1            -- shorthand
1 :: Num a => a
ghci> :t 1::Int       -- explicitly specify
1 :: Int
ghci> :t 1::Double
1 :: Double
```

Usage

```
-- usage: declaration
x :: Int
f x :: Int -> Int

-- usage: expression
x = (1::Int)
x = (Right 1::Maybe a Int)
```

```
-- Basics
--
x :: Int
x,y,z :: Int      -- Multiple
x :: ()           -- unit type i.e. like null
1 :: Char         -- ERROR. Wrong type.
1 :: a            -- ERROR. i.e. cannot be a variable. nullary data
is already defined so cannot be a variable


---- Monomorphic type
-- i.e. a type constructor with no type variables
1 :: Int
1 :: Double


---- Polymorphic type
-- i.e. a type constructor with n type variables
--
-- nary + all variables specified
[1] :: [Int]                  -- unary
Just 1 :: Maybe Int
(1,2) :: (Int,Int)            -- binary
Left 1 :: Either Int Int
Right 1 :: Either Int Int
```

```
-- nary + unspecified variables
-- i.e. type variables
Left 1 :: Either Int a
Left 1 :: Either a b         -- ERROR. a is already bound i.e. 1
Right 1 :: Either a Int

-- nesting
-- i.e. nary within nary
[Just 1] :: [Maybe Int]
[Just 1] :: [] Maybe Int                    -- ERROR. Treated as
[a] b, which is non-sense
Just (Left 1) :: Maybe (Either Int Int)
Just (Left 1) :: Maybe Either Int Int       -- ERROR. Treated as
Maybe a b c  i.e. Maybe with 3 variables
Just (Left 1) :: Maybe (Either Int a)       -- with type variable

-- nesting + unspecified variables in inner
-- e.g. Either a
Data.Functor.Sum.InL :: forall {k} (f :: k -> *) (g :: k -> *)
(a :: k). f a -> Data.Functor.Sum.Sum f g a
InL (Left 1) :: Sum (Either Int) g a     -- OK i.e. Either Int a
InL (Right 1) :: Sum (Either c) g Int    -- OK i.e. Either c Int



---- shorthands: special shorthands for ->, [], (,)
--
-- function shorthand
(*2) :: Int -> Int            -- shorthand
(*2) :: (->) Int Int          -- same
(*) :: Int -> Int -> Int        -- shorthand for n-ary
(*) :: (->) Int ((->) Int Int)  -- same
(*) :: (->) ((->) Int Int) Int  -- ERROR

-- list shorthand
[1] :: [Int]          -- shorthand
[] :: [] Int          -- same

-- tuple shorthand
(1,1) :: (Int,Int)    -- shorthand
(1,1) :: (,) Int Int  -- same


---- Function type signature
--
(*) :: Int -> Int -> Int
(*2) :: Int -> Int               -- partially applied
-- type variables
id :: a -> a

-- type constructor: just a type of function
Just :: a -> Maybe a
Left :: a -> Either a b
Right :: b -> Either a b
```

```
(,) :: a -> b -> (a,b)
(,) :: a -> b -> (,) a b           -- same
(,,) :: a -> b -> c -> (a,b,c)
(,,) :: a -> b -> c -> (,,) a b c  -- same
-- partially applied
(1,) :: b -> (Int,b)
(,1) :: a -> (a,Int)


---- Signature of user defined types
--
-- case: algebraic data type
data MyType = A | B Int
A :: MyType
B 1 :: MyType
data MyType a = A a       -- with type variable
A 1 :: MyType Int

-- case: type alias
type MyType = Int
1 :: MyType
type MyType a = [a]        -- with type variable
[1] :: MyType Int

-- case: newtype
newtype Wrapper m a = A (m a)
A [1] :: Wrapper [] Int
A (Just (Left 1)) :: Wrapper Maybe (Either Int a)
```

Context aka type class signature
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-800004.4

Context aka **type class** signature
Applies only when there are **type variables.**
Specifies classes applying on the variables.

Usage

```
-- usage: expression type signature
x = 1 :: Num a => a
f x = (Num a, Show b) => a -> b

-- usage: class declaration
class (Read a, Show a) => MyClass a where
```

```
--
1 :: Num a => a
1 :: (Num a, Show a) => a             -- multiple i.e. a is a type
implementing class Num + Show
1 :: (Foldable a, Eq a) => a          -- ERROR. Fails because
Foldable=unary type but Eq=nullary type
```

```
                                    -- i.e. Foldable (a::Type ->
Type), Eq (a::Type)
(==1) :: (Num a, Eq a) => a -> Bool   -- type class in a function
1 :: Num Int                        -- ERROR. i.e. don't specify
class instance
1 :: Int                            -- Specify the type instead.
Int implies Num Int
```

Type equality constraint (~)
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/equality_constraints.html

Type equality constraint ~
Check equality of types after they are **resolved**.
e.g. resolve type family F
  F x ~ 'True

Example: **type family/synonym** as a constraint
e.g.

type family IsEven (n :: Nat) :: Bool
f :: IsEven n ~ 'True => ... like even x == True

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE StandaloneKindSignatures #-}

import Data.Fin
    -- fin: https://hackage.haskell.org/package/fin-0.2.1
import Data.Type.Nat

-- IsEven type level function
type IsEven :: Nat -> Bool
type family IsEven n where
  IsEven 'Z = 'True
  IsEven ('S 'Z) = 'False
  IsEven ('S ('S n)) = IsEven n

isEven :: IsEven n ~ 'True => SNat n -> IO ()
    -- equality constraint IsEven n ~ 'True
isEven x = putStrLn $ show (snatToNatural x) <> " is even"

-- e.g.
isEven (SZ)                   -- "0 is even"
isEven (SS @('Z))             -- ERROR
isEven (SS @('S 'Z))          -- "2 is even"
isEven (SS @('S ('S 'Z)))     -- ERROR
```

Example: (a ~ b => MyClass a b) vs (MyClass a a)
e.g. used in Indexable lens
Control.Lens.Indexed: https://hackage.haskell.org/package/lens-5.2/docs/Control-Lens-

Indexed.html#g:1

```
instance i ~ j => Indexable i (Indexed j)            -- YES
instance Indexable i (Indexed i)                      -- NO
```

[SO] equality constraint in instances: https://stackoverflow.com/questions/11553705/haskell-equality-constraint-in-instance
Constraint trick: https://chrisdone.com/posts/haskell-constraint-trick/

Trick based on how GHC defines class instances
GHC **defines** class instances **without constraints**. Constraints are applied only when **resolving** instances.
(MyClass a a) is defined only if the **variables are equal**.
a ~ b => (MyClass a b) is defined for **any variables a and b**, and applies (a ~ b) on resolution

```
class MyClass u v where
  fn :: u -> v
instance MyClass a a                Defined for u, v only if u == v
fn @a                               ERROR. No instance
fn @a @a                            OK

instance a ~ b => MyClass a b       Defined for all u, v
fn @a                               OK  Applies constraint (a ~ b)
```

```
data A a = A a
data B a = B a
data C a = C a

class MyClass u v where
  getMyValue :: u -> v

instance a ~ b => MyClass (A a) b where                -- GOOD
 Instance for any (A u) + v, but valid only when v == u
  getMyValue (A a) = a

instance MyClass (B a) a where                         -- BAD
Instance only for B u + v where v == u
  getMyValue (B a) = a

instance a ~ b => MyClass (C a) [b] where              -- BAD
Instance only for C u + v where v == [u]
  getMyValue (C a) = [a]


-- e.g.
-- OK
x = getMyValue (A True)             -- OK
                                    -- MyClass (A u) v for any
u,v
                                    --  ---> MyClass (A Bool) v
       u == Bool given
                                    --  --> MyClass (A Bool) Bool
```

```
      resolve u ~ v constraint

y = getMyValue (B True)                          -- ERROR No
instance MyClass (B Bool) v yet since v is unknown
                                                 --
message "Ambiguous type variable 'b0' arising from a use of
'getMyValue' prevents the constraint '(MyClass (B Bool) b0)' from
being solved."
y = getMyValue (B True) :: Bool                  -- OK
y = getMyValue @(B Bool) @Bool (B True)          -- OK

z = getMyValue (C True)                          -- ERROR No
instance MyClass (C Bool) v yet since v is unknown
z = getMyValue (C True) :: [Bool]                -- OK
z = getMyValue @(C Bool) @Bool (C True)          -- OK
```

Example: a unique family F a for each a

```
{-# LANGUAGE TypeFamilyDependencies #-}
{-# LANGUAGE FunctionalDependencies #-}
{-# LANGUAGE FlexibleInstances #-}

type family MyValue a = r where
  MyValue Int = [Int]
  MyValue String = (String, String)

class GetMyValue a (MyValue a) | b -> a where              -- ERROR
  ...

class MyValue a ~ b => GetMyValue a b | b -> a where       -- OK
  getMyValue :: b

instance GetMyValue Int [Int] where
  getMyValue = [1]

instance GetMyValue String (String, String) where
  getMyValue = ("a", "b")

x  = getMyValue :: [Int]                                  -- [1]
y  = getMyValue :: (String, String)                       --
("a", "b")


-- Syntax
--
f :: a ~ b => a -> b                  -- function
class a ~ b => MyClass a b            -- class
instance a ~ b => MyClass a b         -- instance

type T a = ...
```

```
type family F a
data D a = ...
f :: a ~ Int => a -> a                -- equal to concrete type
f :: b ~ F a => a -> b                -- equal to type family b = F
a
f :: b ~ T a => a -> b                -- equal to type alias b = T
a
f :: a ~ b => a -> D b                -- equal to type variable
```

==========

GHC extensions

GHC Extension: ExplicitForAll
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/explicit_forall.html

Explicit forall
Variables in a type signature are **free variables** by **default**.
forall is used to make free variables **explicit**.
e.g.

```
id :: a -> a                      a free by default
id :: forall a. a -> a            a free explicitly
```

In particular, this is important for variables **functions** as arguments
e.g.

```
apply :: forall a. (forall b. b -> b) -> a -> a
apply f = f                                          -- apply f :: b
-> b on any input

apply id 0
apply id ""
```

```
---- Basic
--
{-# LANGUAGE ExplicitForAll #-}
f :: a -> a                     -- a=free implicitly by default
f :: forall a. a -> a           -- a=free explicitly
f :: Eq a => a -> a             -- implicit forall i.e. "∀a if Eq a
then ..."
f :: forall a. Eq a => a -> a   -- explicit forall
f :: Eq a => forall a. a -> a   -- BAD Eq a is ignored i.e Same as
                                --    f :: Eq a => forall b. b

-- multiple variables
f :: a -> b -> a                          -- a=forall b=forall
implicit
f :: forall a b. a -> b -> a              -- same explicit
f :: forall a. forall b. a -> b -> a      -- same
f :: forall a. a -> forall b. b -> a      -- same
f :: Eq b => a -> b -> a                  -- a=forall implicit
```

```
b=Eq
f :: forall a. Eq b => a -> b -> a          -- same explicit
f :: Eq b => forall a. a -> b -> a          -- same

-- with kind signatures
-- Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/
kind_signatures.html
{-# LANGUAGE KindSignatures #-}
import Data.Kind
f :: forall (a::Type) (b :: Type). a -> b
f :: forall (a::Type) (b :: Type). Eq b => a -> b

-- with polykind kind signatures
{-# LANGUAGE PolyKind #-}
f :: forall a. a -> a                      -- polykind         i.e.
a ~ Int (wellformed type), Maybe (partially applied), 'True (data
kind)
f :: forall k (a :: k). a -> a             -- same
f :: forall (a :: k) k. a -> a             -- ERROR. Order matters

-- case: polykind
f :: forall a. Proxy a
f :: forall k (a :: k). Proxy a              -- same
f = Proxy

-- e.g.
x = f :: Proxy Int            -- Proxy Int          wellformed
type
x = f :: Proxy Maybe          -- Proxy Maybe        partially
applied type
x = f :: Proxy 'True          -- Proxy 'True        data kind


---- Applications
--
-- case: type signatures
f :: forall a. a -> a
f :: forall a. (Eq a, Show a) => a -> a
f :: (Show a, Eq a) => forall a. a -> a

-- case: class instance
instance MyClass [a] where ...                              --
 forall implicit
instance forall a. MyClass [a] where ...                    --
explicit
instance forall a. (Show a, Eq a) => MyClass [a] where ...

-- case: existential quantification
{-# LANGUAGE ExistentialQuantification #-}
data A = A a                                  -- ERROR
data A = forall a. A a                        -- OK
data A = forall a. (Eq a, Show a) => A a
```

```
-- case: kind signatures
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE RankNTypes #-}
type T :: k -> Type                                          --
forall implicit
type T :: forall k. k -> Type                                --
explicit
type T :: forall k1 k2. k1 -> k2 -> Type                     --
multiple
type T :: forall (k1::Type) (k2::Type). k1 -> k2 -> Type   -- with
restricted kinds
type T :: forall k (a :: k). a -> Type

-- case: type family
{-# LANGUAGE TypeFamilies #-}
type family F :: b -> Type                             -- implicit
forall
type family F :: forall b. b -> Type                   -- explicit
type family F a :: forall b. b -> Type
```

Rule: "forall or nothing rule"
Two types of type variable **defaults**:
1) NO forall at top: type variable is **forall** by default.
i.e. **implicitly** defined as forall.

2) forall at top: type variable is **nothing** by default.
i.e. undefined

e.g.
case: forall at top
i.e. nothing by default

```
f :: forall a b. a -> b -> a              -- a=forall, b=forall
f :: forall a. a -> forall b. b -> a      -- same
f :: Eq b => forall a. a -> b -> a        -- a=forall, b=Eq
f :: forall a. a -> b -> a                -- ERROR. b=nothing. i.e.
is not defined
```

case: no forall at top
i.e. forall by default

```
f :: a -> b -> a                          -- a=forall, b=forall
f :: forall a b. a -> b -> a              -- same
f :: a -> forall b. b -> a                -- same
f :: Eq b => a -> b -> a                  -- a=forall, b=Eq
f :: (forall a. a -> b -> a)              -- a=forall, b=forall
```

GHC extension: lexically scoped variables
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/

scoped_type_variables.html#extension-ScopedTypeVariables

Scoped type variables
Use type variables from **signature** in the **expression** too
i.e. variable has scope in expression
e.g. without extension

```
f :: a -> a
f x = (x::a)         -- ERROR. "Couldn't match expected type 'a1'
with actual type 'a' 'a1' is a rigid type variable ..."
                     - i.e. a in binding (x::a) NOT a in signature
(a -> a)
f :: a -> a         -- i.e. same as above
f x = (x::a1)
```

with extension

```
{-# LANGUAGE ScopedTypeVariables #-}
f :: forall a. a -> a
f x = (x::a)            -- OK
```

e.g. using a constant function

```
getMaxEnum :: forall a. (Enum a, Bounded a) => a -> Int
getMaxEnum _ = fromEnum maxBound                -- ERROR. "Could
not deduce (Enum a0) arising from a use of 'fromEnum' from the
context"
getMaxEnum _ = fromEnum (maxBound::a)           -- OK
getMaxEnum _ = fromEnum (maxBound @a)           -- OK
```

GOTCHA: MUST **explicitly** specify **forall**
Problem: lexically scoping only with **forall** explicitly indicated

```
{-# LANGUAGE ScopedTypeVariables #-}
f :: a -> a             -- ERROR "Couldn't match expected type 'a1'
with actual type 'a' 'a1' is a rigid type variable ..."
f x = (x::a)           -- i.e. a in signature is NOT the same a in
binding
f :: a -> a             - i.e. same as
f x = (x::a1)

f :: forall a. a -> a   -- GOOD. forall a expliclt
f x = (x::a)            -- GOOD
```

GHC extension: Type application
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/type_applications.html#extension-TypeApplications

Type application
Explicitly apply types on type variables

e.g. usage: solving error= "Ambiguous type variable ~ arising from ~"

```
-- e.g. maxBound :: Bounded a => a
x = maxBound             -- ERROR. "Ambiguous type variable a0
arising from maxBound"
x = maxBound @Int        -- x=9223372036854775807

--e.g. read :: Read a => String -> a
x = read "1"             -- ERROR. "Ambiguous type variable a0
arising from read"
x = read @Int "1"        -- x=1
```

```
{-# LANGUAGE TypeApplications #-}
f :: a -> b -> c
f @Bool                   -- f :: Bool -> b -> c
f @Bool @Char             -- f :: Bool -> Char -> c
f @Bool @Char @Double     -- f :: Bool -> Char -> Double

-- application
f @Bool True 'c'
(f @Bool) True 'c'        -- OK, but no need for ()
f @Bool @Char True 'c'
f True @Char 'c'          -- ERROR
f @Bool 1 'c'             -- ERROR 1 not an Bool
f @Bool @Char True 1      -- ERROR 1 not a Char

-- order matters
g :: (Eq c, Eq b, Eq a) => a -> b -> c
g @Bool                  -- g :: a -> b -> Bool
g @Bool @Char            -- g :: a -> Char -> Bool
g @Bool @Char @Double    -- g :: Double -> Char -> Bool


-- case: with kind signature
f :: Proxy (a :: k) -> b
f @Int                       -- ERROR. Need to specify k first.
f @Type @Int                 -- k=Type a=Int
f @(Type -> Type) @Maybe     -- k=Type -> Type  a = Maybe


---- Inferred/specified type variables
--
-- Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/
type_applications.html
{-# LANGUAGE KindSignature #-}
f :: forall a. a -> ...              -- a specifiable
f :: forall {a}. a -> ...            -- a not specifiable i.e.
inferred
```

```
f :: forall k {a :: k}. a -> ...      -- with kind signature
f :: forall a {b} {c}. a -> ...       -- multiple
```

```
f :: forall a b. a -> ...
g :: forall {a} b. a ->
f @Int @Bool       -- a=Int b=Bool
g @Int             -- b=Bool        i.e. a can't be specified
g @Int @Bool       -- ERROR
```

GHC extension: rank polymorphism
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/rank_polymorphism.html

Rank N polymorphism
Allow **free variables** in nested signatures
e.g.

f :: a -> a	Rank 1 (implicit)
f :: forall a. a -> a	same explicit
f :: forall a. Eq a => a -> a	Rank 1
f :: forall a b. a -> b -> a	Rank 1
f :: forall a. a -> forall b. b -> a	Rank 1
f :: (forall a. Eq a => a -> a) -> Int	Rank 2. forall on argument 1 instead of f itself
f :: ((forall a. Eq a => a -> a) -> Bool) -> Bool	Rank 3

usage: specify classes in constructor

```
{-# LANGUAGE RankNTypes #-}

data MyPrinter = MyPrinter (Show a => a -> IO ())                -- ERROR. error="Not in scope: type variable a"
data MyPrinter = MyPrinter (forall a. Show a => a -> IO ())      -- OK

-- e.g.
printInt :: Int -> IO ()
printInt = print
MyPrinter print                     -- OK
MyPrinter (print :: Int -> IO())    -- ERROR error="Couldn't match type a with Int"
MyPrinter printInt                  -- ERROR
```

usage: strictly limit types
e.g. a function which maps
- strictly only **Int** list
- strictly only generic **Num a** functions

```
{-# LANGUAGE RankNTypes #-}

-- BAD: this is what we want, but undefinable as is
mapIntUndefinable :: Num a => (a -> a) -> [Int] -> [Int]
```

```
mapIntUndefinable f l = f <$> l                              --  ERROR
"Couldn't match type a with Int"

-- BAD: map too narrow
mapIntTooNarrow :: (Int -> Int) -> [Int] -> [Int]
mapIntTooNarrow f l = f <$> l

-- BAD: list too generic
mapIntTooWide :: Num a => (a -> a) -> [a] -> [a]
mapIntTooWide f l = f <$> l

-- GOOD: accepts only strictly generic Num functions
mapInt :: (forall a. Num a => a -> a) -> [Int] -> [Int]
mapInt f l = f <$> l

-- e.g.
fNum :: Num a => a -> a
fNum = (* 10)
fInt :: Int -> Int
fInt = (* 100)
listInt :: [Int]
listInt = [1 .. 10]
listDouble :: [Double]
listDouble = [1.1,1.2,1.3]

-- BAD. Accepts Int only functions.
mapIntTooNarrow fNum listInt
mapIntTooNarrow fInt listInt            -- BAD
-- BAD. Accepts Double list.
mapIntTooWide fNum listInt
mapIntTooWide fNum listDouble           -- BAD
-- GOOD
mapInt fNum listInt                     -- OK
mapInt fInt listInt                     -- ERROR as expected. fInt
is Int only.
mapInt fNum listDouble                  -- ERROR as expected. Int
list only.
```

GHC extension: Impredicative types
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/impredicative_types.html
Wiki: https://wiki.haskell.org/Impredicative_types

Impredicative types
Allow **rank polymorphism** in type definitions as well.
e.g.

Maybe (forall a. [a] -> [a])

GHC extension: ambiguous types
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/

ambiguous_types.html

Ambiguous type
A **function type signature** is ambiguous if the function cannot be called without explicit type specification.

usage: ambiguous class function
e.g. labelling with a class
WIthout AllowAmbiguousType

```
class MyLabel a where
  myLabel :: String           -- ERROR "Could not deduce (MyLabel
a0) arising from a type ambiguity check
                              --   for the type signature for
'myLabel' from the context:
                              --   MyLabel a bound by the type
signature for: myLabel :: forall {k} (a :: k)."
```

2 solutions:
a) use **proxy**
b) use **AllowAmbiguousType**

Solution a) with Proxy

```
import Data.Proxy

type MyLabel :: forall a. a -> Constraint
class MyLabel a where
  myLabel :: Proxy a -> String
instance MyLabel String where myLabel _ = "str"
instance MyLabel Maybe where myLabel _ = "maybe"

-- e.g.
myLabel (Proxy @String)        -- "str"
myLabel (Proxy @Maybe)         -- "maybe"
```

Solution b) with AllowAmbiguousType

```
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE TypeApplications #-}
import Data.Kind

type MyLabel :: forall a. a -> Constraint
class MyLabel a where
  myLabel :: String

instance MyLabel String where myLabel = "str"
instance MyLabel Maybe where myLabel = "maybe"

-- e.g.
myLabel @String        -- "str"
myLabel @Maybe         -- "maybe"
```

GHC Extension: Type families
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_families.html

Type families (indexed types)
A mapping from type to type. i.e. a **function** for types.
Allows a type to be **computed** by a function.

3 types of maps
1) **type synonym family:** computed type synonym
 1a) **open** family: mappings can be defined dynamically
 1b) **closed** family: opposite
2) **data family**: compute a new algebraic data type
3) **associated** families: computed type and data families per class instance

Associated type vs type families
When to use?

a) **associated type**: simple type alias

e.g. Servant

```
class HasServer api context where
  type ServerT api (m :: Type -> Type) :: Type
  ...

instance (HasServer a ctx, HasServer b ctx) =>  HasServer (a :<> b)
ctx where
  type ServerT (a :<> b) m = ServerT a m :<> ServerT b m
instance (HasServer api ctx, KnownSymb p) => HasServer (p :> api)
ctx where
  type ServerT (p :> api) m = ServerT api m
...
```

b) **open type family**:
Associated type, but if it needs to be **shared** (e.g. use in multiple type classes)
e.g. Lens

```
module Control.Lens.At

type family Index (s :: Type) :: Type          -- index of a
container
type instance Index [a] = Int
type instance Index (a, b) = Int
type instance Index (a -> b) = a
type instance Index Text = Int
type instance Index ByteString = Int
type instance Index (Map k a) = k
...
```

```
type family IxValue (m :: Type) :: Type        -- value of a
container
type instance IxValue [a] = a
type instance IxValue (a, b) = a
type instance IxValue (a -> b) = b
type instance IxValue Text = Char
type instance IxValue ByteString = Word8
type instance IxValue (Map k a) = a
...


-- e.g. typeclass
class Ixed m where
 ixed :: Index m -> Traversal' m (Maybe (IxValue m))

class Contains m where
 contains :: Index m -> Lens' m Bool
```

e.g. Singleton

```
module Data.Singleton

type Sing :: forall k. k -> Type
type family Sing

type SBool :: Bool -> Type
data SBool a where
  STrue :: SBool 'True
  SFalse :: SBool 'False

type SMaybe :: forall k. Maybe k -> Type
data SMaybe k where
  SNothing :: SMaybe 'Nothing
  SJust :: Sing x -> SMaybe ('Just x)

type instance Sing = SBool
type instance Sing = SMaybe
...
```

c) **closed type family**: type **function**

e.g. Prelude.Singleton

```
module Prelude.Singleton

type Not :: Bool -> Bool
type family Not a where
  Not 'True = 'False
  Not 'False  = 'True
```

```
type If :: Bool -> k -> k -> k where
  type If 'True x y = x
  type If 'False x y = y

...
```

usage: open type synonym

```
{-# LANGUAGE TypeFamilies #-}
import Data.Word
import Data.Kind

-- step) define type
type family Simplify (a :: Type) -> Type
type instance Simplify Int = Word8
type instance Simplify Double = Word8
type instance Simplify Bool = Char
type instance Simplify (Maybe a) = String

-- step) define values
class CanSimplify a where
  simplify :: a -> Simplify a
instance CanSimplify Int where simplify = fromIntegral
instance CanSimplify Double where simplify = round
instance CanSimplify Bool where simplify True = 'T'; simplify False
= 'F'
instance CanSimplify (Maybe a) where simplify _ = "Maybe"

-- e.g.
simplify (1::Int)            -- 1 :: Word8
simplify (1.2::Double)       -- 1 :: Word8
simplify True                -- 'T'
simplify (Just ())           -- "Maybe"
```

similar with closed family

```
type family Simplify2 (a :: Type) :: Type where
  Simplify2 Int = Word8
  Simplify2 Double = Word8
  Simplify2 Bool = Char
  Simplify2 (Maybe a) = String
  Simplify2 a = String                    -- IMPORTANT: catch all
is possible in closed family only
```

usage: associated class

```
{-# LANGUAGE TypeFamily #-}
import Data.Word
import Data.Kind
```

```
-- step: define class
class Graph g where
  type Vertex g = v
  data Edge g
  src, tgt :: Edge g -> Vertex g
  outEdges :: g -> Vertex g -> [Edge g]
  inEdges :: g -> Vertex g -> [Edge g]

-- step: define instance
newtype GGraph a = GGraph [Edge (GEdges a)] deriving (Show, Eq)
instance Eq a => Graph (GGraph a) where
  type Vertex (GGraph a) = a
  data Edge (GGraph a) = GEdge (Vertex (GGraph a)) (Vertex (GGraph
a))
    deriving (Show, Eq)
  src (GEdge v _) = v
  tgt (GEdge _ v) = v
  outEdges (GGraph l) v = filter ((== v) . src) l
  inEdges (GGraph l) v = filter ((== v) . tgt) l

-- e.g.
g = GGraph [
  GEdge 1 2,
  GEdge 1 3,
  GEdge 1 4
    ]
src $ GEdge 1 2       -- 1
tgt $ GEdge 1 2       -- 2
outEdges 1            -- [GEdge 1 2, GEdge 1 3, GEdge 1 4]
inEdges 2             -- [GEdge 1 2]
```

Basics

```
---- Basics
{-# TypeFamilies #-}
import Data.Kind
type F :: Type -> Type
type family F a
type family F (a :: Type) :: Type      -- same. i.e. monomorphic
param by default

type F :: Type -> Type -> Type
type family F a b                      -- multiple parameters
 e.g. type instance F Int Bool = Char

type F :: (Type -> Type) -> Type
type family F (a :: Type -> Type)      -- polymorphic param
e.g. type instance F Maybe = Char
type family F a :: Type -> Type        -- polymorphic result
 e.g. type instance F Int = Maybe
```

```
type F :: (Type -> Type) -> Type -> Type -> Type -> Type
type family F (a :: Type -> Type) (b :: Type) :: Type -> Type ->
Type    -- combined.     e.g. type instance F Maybe Int = Either


-- case: no parameters
-- i.e. exactly one instance
type F :: Type
type family F
type family F :: Type                  -- same              e.g.
type instance F = Int

type F :: Type -> Type
type family F :: Type -> Type          -- polymorphic       e.g.
type instance F = Maybe

type F :: forall k. k -> Type
type family F :: k -> Type             -- polykind          e.g.
type instance F = Proxy


-- case: kind polymorphism
type F :: forall k. k -> Type
type family F (a :: k)             -- polymorphic param

type F :: forall k. Type -> k
type family F a :: k               -- polymorphic output

type F :: forall k l. k -> l
type family F (a :: k) :: l        -- polymorphic + independent

type F :: forall k. k -> k
type family F (a :: k) :: k        -- polymorphic + dependent

-- case: polykind in parameter
type family F (a :: k)
type instance F Int = Bool            -- OK
type instance F Maybe = Bool          -- OK
type instance F Int = Maybe           -- ERROR

-- case: polykind in result
type family F a :: k
type instance F Int = Bool            -- Ok
type instance F Int = Maybe           -- OK
type instance F Maybe = Bool          -- ERROR

-- case: polykind in both independently
type family F (a :: k) :: l
type instance F Int = Bool             -- OK
type instance F Int = Maybe            -- OK
type instance F Maybe = Bool           -- OK
type instance F Maybe = Maybe          -- OK

-- casae: polykind in both dependently
```

```
type family F (a :: k) :: k
type instance F Int = Bool             -- both monomorphic
type instance F Maybe = []             -- both unary
type instance F Int = []               -- ERROR
type instance F Maybe = Bool           -- ERROR


-- case: forall
type family F a :: forall k. k -> Type      -- k is a phantom

-- case: phantom type
type family F a :: forall k. k -> Type
data X a = X
data Y a = Y a
type instance F X        -- OK
type instance F Proxy    -- OK
type instance F Y        -- ERROR


---- Instances
--
-- case: open type synonym
type family F a
type instance F Int = Bool
type instance F (Maybe a) = Bool

-- explicit kind param
type family F (a :: k)
type instance F Int = Bool
type instance F @Type Int = Bool               -- explicit k = Type
type instance F Maybe = Bool
type instance F @(Type -> Type) Maybe = Bool   -- explicit k = Type
-> Type
type instance F 'True = Bool
type instance F @Bool 'True = Bool             -- explicit k = Bool


-- case: close type synonym
type family F a where
 F Int = Bool
 F (Maybe a) = Bool
 forall b. F (Maybe b) = Bool        -- same with forall

 F b = Bool             -- catchall
 forall b. f b = Bool   -- same with explicit forall


-- case: data family
data family F a
data instance F Int = MyIntF Int        -- instance by new
algebraic type
newtype instance F Int = MyIntF Int     -- instance by newtype
```

```
data family F a b
data instance F Int b = MyIntF b                -- with variable
data instance forall b. F Int b = MyIntF b      -- explicit forall


-- case: associated class
class MyClass a where
 type F a
 type F a :: Type
 data G a
 data G a :: Type
instance MyClass Int where
 type F Int = Bool
 data G Int = MyG String
```

GHC extension: type family dependencies
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_families.html#type-families-and-instance-declarations

Dependencies aka **injectivity**
Ensure that type family is **injective**. i.e. type family is invertible

type family F a b c = r | r -> a b	if F a b * = r, then a=a0 b=b0
type instance F () () () = ()	OK. if F a b c = (), then a=() b=()
type instance F () () Int = ()	OK
type instance F () Int () = ()	ERROR
type instance F Int Int () = ()	ERROR

Needed to resolve **ambiguities** when the type family needs to be inverted
e.g.

```
-- without dependencies
type family A a = r
type instance A Int = Int
type instance A Bool = Int

-- with dependencies
type family B a = r  | r -> a     -- if B a = r, then a=a0
type instance B Int = Int         -- if B a = Int, then a = Int
type instance B Bool = Int        -- ERROR.
type instance B Bool = Bool       -- OK

type family C a = r
type instance C Int = Int
type instance C Bool = Bool


fn :: A a -> C a             -- ERROR. Cannot determine a
                             -- e.g. if A a = Int, then a = Int or
Bool?
fn :: B a -> C a             -- OK. Can determine a
                             -- e.g. if A a = Int, then a = Int
```

```
type family F a = r | r -> a
type family F a b = r | r -> a
type family F a b = r | r -> a b    -- dependent on multiple
variables
type family F a = r | a -> r        -- ERROR
```

```
type family F a = r | r -> a
type instance F Int = Boo           -- OK
type instance F Char = Bool         -- ERROR

type family F a b = r | r -> a
type instance F Int Int = Bool       -- dependency r = Bool  ->  a
= Int
type instance F Int Char = Bool      -- OK
type instance F Char Int = Bool      -- ERROR

type family F a b = r | r -> a b
type instance F Int Int = Bool       -- dependency r = Bool  ->  a
= Int b=Int
type instance F Int Char = Bool      -- ERROR
type instance F Char Int = Bool      -- ERROR
```

GHC extension: Type operators
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/type_operators.html

Type operators
Use operators in type names

e.g. usage: algebraic sum and product with operators

```
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE NoStarIsType #-}

data a + b = Inl !a | Inr !b deriving (Show)      -- Algebraic sum
type
data a * b = !a :*: !b deriving (Show)            -- Product type

infixl 6 +                              -- i.e. a + b + c = (a + b)
+ c
infixl 7 *

x = Inl (Inl 1) :: Int + a + b
x = Inl (Inl 1) :: (Int + a) + b        -- same
y = Inr (Inl 1) :: a + Int + b
z = Inr 1 :: a + b + Int
```

Define point

```
type Point a = a + a * a + a * a * a              -- 1/2/3
dimensional point
type Point a = (a + (a * a)) + ((a * a) * a)      -- same

-- e.g.
p1 = Inl $ Inl 1 :: Point Int                    -- 1D point
p2 = Inl $ Inr $ 1 :*: 2 :: Point Int            -- 2D point
p3 = Inr $ 1 :*: 2 :*: 3 :: Point Int            -- 3D point
```

GHC Extension: role annotations
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/roles.html#extension-RoleAnnotations

data T a = T — a = phantom

data T a = T a — a = representational since a1 ~ a2 => T a1 ~ T a2

data T a = T (Maybe a) — a = representational since a1 ~ a2 => Maybe a1 ~ Maybe a2 => T a1 ~ T a2

data T a = T (AFamily a) — a = nominal since a1 ~ a2 does NOT imply AFamily a1 ~ AFamily a2

e.g.

```
type Age = Int                          Int ~ Age
type family AFamily where               AFamily Int !~ AFamily Int
  type AFamily Int = Char
  type AFamily Age = Bool
```

data T m a = T (m a) — m = representational a = nominal

```
---- Basic
--
-- role types
data T a = T                      -- a=phantom
data T a = T a                    -- a=representational
data T a = T (Maybe a)            -- a=representational
data T a = T (SomeTypeFamily a)   -- a=nominal
data T m a = T (m a)              -- m=representational a=nominal
data T a b m c = T b (m c)        -- a=phantom b=representational
m=representational c=nominal

-- role annotations
data T a m b = T b (m b)
type role T1 phantom representational nominal   -- a=phantom
m=representational b=nominal
type role T1 _ _ _                              --
a,m,b=unspecified
type role T1 phantom _ _                        -- a=unspecified
type role T1 _ representational nominal
```